SENTINELMIND AI v2.0

Individual Responsibilities & Comprehensive Viva Defense Guide

Institution: Acharya Institute of Technology — Department of Artificial Intelligence & Machine Learning
Project: Multilingual Sarcasm, Threat Intelligence & Safety Guard Engine
Purpose: Complete Technical Documentation of Individual Team Roles, Architectural Contributions & Viva Q&A;

1. Master Responsibilities Matrix

The SentinelMind AI project is structured into four specialized pillars spanning neural transformer research, multilingual data engineering, full-stack application development, and AI harm moderation literature. The following matrix summarizes the core ownership of each team member as presented to the evaluation committee:

Team Member	Major Assigned Responsibilities	Key Codebase Modules & Deliverables
Abhiraju R	System Architecture, Transformer-based NLP Framework, Backend/API Development, Multimodal Integration	app.py, models/deberta_engine.py, models/fusion_engine.py, PDF/OCR Ingestion
Adarsh KN	Dataset Analysis, Data Preprocessing, Multilingual/Code-Mixed Text Handling, Model Research	language_detector.py, train_model.py, models/feature_extractor.py, Indic Lexicons
Ayush H Mane	Frontend Development, System Testing, Research-Gap Analysis, Result Analysis, Project Documentation	frontend/, index.html, test_e2e_features.py, Glassmorphism UI & Benchmarks
Bharath CN	Literature Review, Dataset Analysis, Multilingual Hate Speech Research, Harm Moderation & Documentation	toxicity_model.py, explainability.py, Policy Compliance, Multi-Category Threat Rules

2. Abhiraju R — System Architecture & Transformer NLP

Role: System Architecture, Transformer-based NLP Framework, Backend/API Development, and Integration.

A. Deep Technical Description of Responsibilities

• **System & Pipeline Architecture:** Engineered the hybrid multi-model inference pipeline. Unified Microsoft DeBERTa-v3 zero-shot NLI, RoBERTa GoEmotions, a custom Scikit-learn multi-task classifier, and heuristic semantic contrast detection under an intelligent Decision Fusion Engine (models/fusion_engine.py).

• **Transformer NLP Framework:** Configured and optimized MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli for zero-shot hypothesis formulation (sarcastic mockery vs sincere discourse) and SamLowe/roberta-base-go_emotions for 28-class fine-grained emotion distribution.

• **High-Performance Backend & API:** Developed the asynchronous FastAPI engine (app.py) running over Uvicorn. Designed the unified POST /analyze endpoint with multi-threaded concurrent translation via ThreadPoolExecutor.

• **Multimodal Extraction Pipeline:** Integrated PyMuPDF for direct PDF binary stream parsing and pytesseract + Pillow for OCR image extraction, feeding parsed text directly into the NLP classification pipeline.

B. Key Technologies & Modules Owned

Tech Stack: Python 3.12, FastAPI, Uvicorn, HuggingFace Transformers, PyTorch, PyMuPDF, Pytesseract, ThreadPoolExecutor.
Core Files: app.py, models/deberta_engine.py, models/fusion_engine.py, download_deberta.py.

C. Key Challenges & Architectural Solutions

• *Challenge:* Transformer latency bottleneck when running NLI zero-shot inference, GoEmotions, and 5-language translation per request.
• *Solution:* Implemented concurrent asynchronous thread-pooling for translation, lazy local weight caching for transformers, and lightweight heuristic gating to bypass heavy neural inference for clear literal/threat cases.

D. Anticipated Viva / Examiner Questions & Model Answers

Q1: Why did you choose Microsoft DeBERTa-v3 instead of standard BERT or RoBERTa?

Answer: DeBERTa-v3 (Decoding-enhanced BERT with Disentangled Attention) uses two separate vectors for each token: one for content and one for relative position. This disentangled attention significantly enhances natural language inference (NLI) and syntactic reasoning, allowing our zero-shot hypotheses to accurately detect complex semantic inversion and irony that standard BERT misses.

Q2: How does your Hybrid Decision Fusion Engine resolve conflicting model predictions?

Answer: The Decision Fusion Engine (models/fusion_engine.py) implements a hierarchical arbitration system. Physical threats and severe toxicity take absolute quarantine priority. For sarcasm, it checks semantic contrast (positive praise + catastrophic outcome). When contrast is confirmed, it overrides surface neural emotion (e.g. converting 'admiration' into 'mockery / sarcastic disapproval') with 98.5% confidence.

Q3: How does the backend process PDF and Image inputs without saving temporary files to disk?

Answer: We handle file uploads in-memory using byte streams. For PDFs, raw bytes are passed directly into pymupdf.open(stream=raw, filetype='pdf') to extract text page-by-page. For images, bytes are converted via io.BytesIO(raw) and processed through pytesseract.image_to_string(), ensuring zero disk I/O overhead and maximum security.

Q4: How did you prevent multi-lingual translation calls from creating a severe response delay?

Answer: We used Python's concurrent.futures.ThreadPoolExecutor to dispatch translation requests for English, Kannada, Tamil, Hindi, and Telugu simultaneously in parallel worker threads, reducing total translation latency from over 4 seconds sequentially to under 600ms concurrently.

3. Adarsh KN — Dataset Engineering & Multilingual Code-Mix NLP

Role: Dataset Analysis, Data Preprocessing, Multilingual/Code-Mixed Text Handling, and Model Research.

A. Deep Technical Description of Responsibilities

- **Multilingual & Code-Mixed Identification:** Engineered language_detector.py, featuring a dual-stage detection mechanism: (1) Unicode range evaluation for native Indic scripts (Devanagari, Kannada, Tamil, Telugu, Malayalam, Bengali, etc.), and (2) Token-density analysis for Romanized code-mixed dialects (Kanglish, Tanglish, Hinglish, Tenglish, Manglish).
- **Dataset Compilation & Stratification:** Curated and balanced the multi-task training corpus (data/training_dataset.csv) covering English, Hinglish, Kanglish, and Tanglish across Sarcasm (52%), Toxic/Abuse (18%), Physical Threat (14%), and Sincere Positive/Literal Communication (16%).
- **Linguistic Feature Engineering:** Authored LinguisticAndContrastExtractor (models/feature_extractor.py), extracting semantic contrast flags, hyperbolic praise density, punctuation intensity ('!?', '...'), capitalization ratios, and word-shape features.
- **ML Model Pipeline Research:** Researched and trained multi-task Scikit-Learn pipelines (TF-IDF character/word n-grams + calibrated classifiers) achieving fast sub-10ms inference for baseline sarcasm and safety verification.

B. Key Technologies & Modules Owned

Tech Stack: Python, Pandas, Scikit-Learn, Joblib, Regex Unicode Parsing, TF-IDF Vectorization.
Core Files: language_detector.py, train_model.py, models/feature_extractor.py, data/training_dataset.csv.

C. Key Challenges & Architectural Solutions

- *Challenge:* Code-mixed Romanized Indic text has non-standardized phonetic spelling (e.g., 'maga' vs 'magane', 'bahut' vs 'bohot', 'kelsa' vs 'kelasa') and overlaps with English stopwords.
- *Solution:* Built indigenous Romanized token lexicons with zero English contamination, combined with code-mix normalization dictionaries and morphological prefix/suffix heuristics.

D. Anticipated Viva / Examiner Questions & Model Answers

Q1: Why is code-mixed language detection particularly challenging for traditional NLP libraries?

Answer: Standard NLP libraries (like standard langdetect or NLTK) rely on native script character frequencies or dictionary lookups. In code-mixed Romanized text (e.g. Kanglish or Hinglish), Indian languages are written in Latin alphabet with mixed English words and non-standard phonetic variations. Our custom token-density extractor in language_detector.py specifically inspects indigenous token sets to accurately identify Kanglish, Tanglish, Hinglish, Tenglish, and Manglish.

Q2: What features were engineered in your custom Scikit-Learn feature extractor?

Answer: In LinguisticAndContrastExtractor, we engineered: (1) Semantic Contrast Collision (praise terms paired with destructive terms), (2) Exclamation and question mark repetition density, (3) Intensifier and hyperbole counts ('super', 'real', 'genius'), (4) Uppercase emphasis ratios, and (5) TF-IDF word and character n-grams (1-3 ngrams) to capture sub-word dialect cues.

Q3: How was the training dataset curated and annotated?

Answer: The dataset was curated from authentic social media discourse, public benchmarks (DravidianLangTech, HASOC), and synthetic conversational samples. Each entry is multi-labeled with: text, language, dialect, sarcasm status, primary emotion, content safety category (safe, toxic, threat), and severity risk level.

Q4: How does the system normalize code-mixed text before translation?

Answer: We built CODE_MIX_GLOSSARY containing slang mapping (e.g., 'maga' -> 'bro/friend', 'sakkat' -> 'super', 'kamaal' -> 'masterpiece'). The pre-normalizer substitutes these domain tokens to provide clean semantic context to the machine translation API.

4. Ayush H Mane — Frontend Studio, System Testing & Research Gap

Role: Frontend Development, System Testing, Research-gap Analysis, Result Analysis, and Project Documentation.

A. Deep Technical Description of Responsibilities

• **Interactive Frontend Studio UI:** Designed and developed the SentinelMind AI Web Dashboard using React (frontend/src/App.js) and standalone Vanilla HTML/CSS (index.html). Implemented a dark-mode glassmorphic interface with 4 dedicated analytical tabs.

• **4-Engine Multi-Model Visualization:** Created real-time breakdown cards showcasing individual engine confidences: Microsoft DeBERTa-v3, Trained ML Pipeline, Semantic Contrast Rules, and the Unified Decision Fusion Engine.

• **End-to-End System Testing & Benchmarking:** Authored automated test suites (test_e2e_features.py, test_ml_fast.py, test_accuracy.py), verifying multilingual code-mixing accuracy, OCR file upload flows, latency, and safety protocols.

• **Research Gap & Result Analysis:** Conducted comparative benchmarking against existing monolingual sentiment tools. Evaluated model precision, recall, F1-scores, and calibrated confidence metrics across 7 diverse real-world test scenarios.

B. Key Technologies & Modules Owned

Tech Stack: React, JavaScript (ES6+), Vanilla CSS (Glassmorphism), Lucide/FontAwesome, Python TestClient, Pytest.
Core Files: frontend/src/App.js, frontend/src/App.css, index.html, test_e2e_features.py, test_ml_fast.py.

C. Key Challenges & Architectural Solutions

• *Challenge:* Presenting dense multi-model AI metrics (translation matrix, 28 emotions, 5 safety meters, explainability text, code-mixed glossary) without cluttering the user interface.
• *Solution:* Implemented modular tabbed navigation (Overview, Translation Matrix, Moderation & Safety, JSON Dev) paired with animated meter progress bars and interactive word-level token highlight tags.

D. Anticipated Viva / Examiner Questions & Model Answers

Q1: What specific research gap did your project identify and address?

Answer: Existing commercial moderation and sentiment tools (like standard AWS Comprehend or Google Cloud Natural Language) suffer from three critical research gaps: (1) They fail completely on Indian code-mixed Romanized dialects like Kanglish and Tanglish, (2) They misclassify sarcastic praise as genuine positive sentiment, and (3) They provide black-box labels without explainable reasoning on why sarcasm or risk was flagged. SentinelMind AI directly solves all three.

Q2: How does the frontend communicate with the FastAPI backend?

Answer: The frontend issues asynchronous multipart/form-data POST requests via Axios/Fetch to /analyze. It supports both raw text payloads and optional attached binary files (PDF/Images). The response JSON is reactively bound to state hooks, instantly updating confidence meters, translation matrices, and dynamic narrative summaries.

Q3: What testing methodologies did you implement to ensure system reliability?

Answer: In test_e2e_features.py, we implemented automated end-to-end integration testing using FastAPI's TestClient. The suite tests 7 benchmark cases: English Sarcasm, Kanglish Sarcasm, Tanglish Sarcasm, Hinglish Sarcasm, Violent Threats, Toxic Profanity, and Kannada Native Script, asserting 100% compliance on language identification, engine breakdown, and safety status.

Q4: How are confidence scores calibrated across the 4 individual engines?

Answer: In our results analysis, individual engine outputs are normalized to a 0.0–100.0% scale. When semantic contrast is confirmed, heuristic confidence is set to 94.0–98.5%. For literal sincere text, confidence is calibrated to reflect certainty of safe authentic communication (e.g. 90.0%), ensuring users receive intuitive, interpretable metrics.

5. Bharath CN — Literature Review & Multilingual Harm Moderation

Role: Literature review, Dataset analysis, multilingual hate speech research, and documentation.

A. Deep Technical Description of Responsibilities

- **Comprehensive Literature Review:** Surveyed leading research in computational sarcasm detection, NLP multi-task learning, and offensive language identification in Dravidian and Indic languages (HASOC, DravidianLangTech at FIRE/EACL/ACL conferences).
- **Multilingual Harm Moderation & Safety Guardrails:** Authored toxicity_model.py, establishing a 5-category safety protocol: (1) Threat & Physical Hazard, (2) Hostility & Toxic Sentiment, (3) Profanity & Vulgarity, (4) Hate Speech & Attacks, and (5) Harassment & Targeted Insult.
- **Multilingual Threat & Abuse Lexicon Engineering:** Built specialized lexicons covering English and Romanized Indic abuse, violent threat keywords, and hate speech triggers across Kannada, Tamil, and Hindi.
- **Dynamic Explainability & Narrative Generation:** Co-developed explainability.py, creating the multi-sentence Executive Intelligence Summary, Semantic Contrast Rationale, Emotion Inversion Note, and Actionable Guidance.

B. Key Technologies & Modules Owned

Tech Stack: Python, Regex Token Sanitization, Multi-Category Moderation Rules, Academic Literature Surveys.
Core Files: toxicity_model.py, explainability.py, Technical Documentation, Policy Taxonomies.

C. Key Challenges & Architectural Solutions

• *Challenge:* Distinguishing between playful sarcastic irony (which is harmless) versus malicious sarcasm masking toxic abuse or violent threats.
• *Solution:* Implemented strict safety precedence in toxicity_model.py: any presence of physical danger triggers automatic quarantine, while non-toxic sarcasm is classified with low risk and safe moderation approval.

D. Anticipated Viva / Examiner Questions & Model Answers

Q1: What were the key findings from your literature review on sarcasm detection?

Answer: Our literature review revealed that sarcasm fundamentally relies on pragmatic incongruity—an intentional clash between literal positive sentiment and negative reality. Prior works (e.g., Maynard & Greenwood, Ghosh et al.) demonstrated that purely lexicon-based or standalone neural models fail without context. This justified our multi-tier architecture combining DeBERTa NLI with explicit semantic contrast heuristics.

Q2: How does the system evaluate and score the 5 safety guardrail categories?

Answer: In toxicity_model.py, text tokens are checked against curated multilingual lexicons and ML threat predictions. Scores are calculated for: Threat (0-99%), Profanity (0-99%), Hate Speech (0-99%), Harassment (0-99%), and Hostility (0-99%). If threat score >= 60%, the system assigns CRITICAL risk and triggers 'BLOCKED / QUARANTINED' action.

Q3: How does the Sanitized & Filtered text generation work?

Answer: Through the censor_text() function in toxicity_model.py, detected profanities and identity attack tokens are dynamically sanitized using regular expression boundary matching, replacing offensive characters with asterisks (e.g., 'f******') while preserving grammatical readability.

Q4: What are the real-world deployment applications of this safety guardrail?

Answer: Applications include: (1) Automated content moderation for regional Indian social media platforms, (2) Brand sentiment analysis (preventing sarcastic complaints from being counted as 5-star praise), and (3) Customer support ticketing prioritization for high-risk or toxic escalations.

6. Cross-Functional Team Synergy & System Flow

SentinelMind AI succeeded through seamless collaboration across all four specialized domains. The diagram and summary below illustrate how individual responsibilities directly feed into one unified production engine:

Stage	Contributing Member	Module & Action Taken	Output Produced
1. Input & UI	Ayush H Mane	React Studio UI / File Uploader	Text, PDF or Image upload
2. Extraction & Ingestion	Abhiraju R	FastAPI /analyze + PyMuPDF / OCR	Clean In-Memory UTF-8 Text Stream
3. Dialect & Translation	Adarsh KN	language_detector.py + Code-Mix Matrix	Dialect Identified + 5-Language Matrix
4. Multi-Tier AI Analysis	Abhiraju R & Adarsh KN	DeBERTa-v3 NLI + GoEmotions + Trained ML	Raw Emotion & Sarcasm Probability
5. Harm Guard & Policy	Bharath CN	toxicity_model.py 5 Safety Meters	Risk Level (Low/Med/High/Crit) + Masking
6. Decision Fusion & XAI	Abhiraju R & Bharath CN	fusion_engine.py + explainability.py	Final Verdict + Narrative Summary
7. Testing & Verification	Ayush H Mane	test_e2e_features.py Benchmark Suite	100% Passed Test Cases & Calibrated UI

7. Key Viva Presentation Tips for the Team

1. **Emphasize the Novelty:** Always highlight that SentinelMind AI is not just a standard English classifier—it solves code-mixing in Indian languages (Kanglish, Tanglish, Hinglish) and provides human-interpretable explainability.

2. **Explain Decision Fusion Clearly:** If asked why you didn't rely purely on deep learning, explain that semantic contrast heuristics + decision fusion protect against neural hallucination and zero-shot ambiguity.

3. **Refer Directly to Code Modules:** When speaking, mention the exact module you owned (e.g., Abhiraju: app.py / fusion_engine.py, Adarsh: language_detector.py, Ayush: test_e2e_features.py / App.js, Bharath: toxicity_model.py).

4. **Highlight Multimodal Capabilities:** Remind examiners that the system handles not only text, but direct PDF reports and OCR image documents in-memory.